

数据表示与运算

有限位宽怎样保持可解释，并显式暴露信息丢失

408 · 计算机组成原理 Topic CO-01

母问题：同一串比特怎样得到确定的数学与硬件结果？

比特没有自带 signed、float 或 address 语义。完整推理链是

Bit Pattern $\rightarrow$ Interpretation $\rightarrow$ Exact Value
 $\rightarrow$ Operation $\rightarrow$ Finite-width Result $\rightarrow$ Lost Information / Flags

任何答案都应先固定 (width, interpretation, operation, destination); 溢出、舍入和异常都发生在“精确数学对象重新进入有限可表示集合”的边界。

本册定位与 Owner 边界

- **Owns:** 位权、定长整数解释、原码/补码、扩展/截断/移位、carry/overflow、乘除基本算法、目标位宽判溢出、IEEE 754 分类/运算/舍入。
- **Uses:** 题设或 ISA 给出的结果宽度、异常语义、舍入模式和状态位定义。
- **Stops:** 具体指令编码和软件可见异常归 CO-02，完整 ALU/控制通路归 CO-03，高级语言规则归机器级映射接口。

目录

1 表示系统：位串、解释与可表示集合	3
2 定长加减：同一低位结果，不同溢出解释	3
3 扩展、截断与移位都在选择保留什么	3
4 乘法：完整乘积、部分积与目标宽度	4
4.1 移位加法与原码一位乘	4
4.2 Booth 的生成逻辑	4
5 除法：商余关系是最终不变量	4
6 浮点：在非均匀格点上表示实数近似	4
7 浮点运算链与舍入证据	5
8 五列概念边界	5

9 做题控制协议	5
10 全科坐标与跨册交接	6
11 考纲映射与来源处理	6

1 表示系统：位串、解释与可表示集合

一个 n 位位串只是集合 $\{0,1\}^n$ 的元素。unsigned 把它解释为

$$U(b) = \sum_{i=0}^{n-1} b_i 2^i, \quad 0 \leq U \leq 2^n - 1;$$

补码把最高位赋负权：

$$T(b) = -b_{n-1} 2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i, \quad -2^{n-1} \leq T \leq 2^{n-1} - 1.$$

同一位串可同时作为无符号数、有符号数、掩码、地址或浮点字段；意义来自消费它的操作，而不是某一位的外观。

原码把符号与绝对值分开，存在 $+0/-0$ ，加减需额外处理符号和大小。补码把定长加减统一到模 2^n 环，只有一个零，代价是范围不对称：最小负数没有同宽正数。

2 定长加减：同一低位结果，不同溢出解释

n 位加法器保留

$$r = (a + b) \bmod 2^n.$$

unsigned overflow 关注最高位 carry；signed overflow 关注精确结果是否落在补码范围。加法中“同号输入得到异号输出”可判 signed overflow；等价电路判据是符号位的进位与出位异或。

4 位运算	unsigned 解释	补码解释
1111+0001 → 0000	15 + 1 越界，carry=1	-1 + 1 = 0，无 overflow
0111+0001 → 1000	7 + 1 = 8，无 unsigned 越界	7 + 1 越界，overflow=1
1000 取负仍为 1000	位模式变换结果	-8 的同宽正数不可表示

减法先明确硬件采用 $a + (\bar{b} + 1)$ ，再按题目对 carry/borrow 的定义解释。零、负、carry、overflow 是不同谓词；不能由一个标志推出另一个解释。

3 扩展、截断与移位都在选择保留什么

zero extension 保留 unsigned 值；sign extension 保留补码值。截断到低 k 位相当于模 2^k ，通常不保留数学值。判断窄化是否无损的通用办法是：截断后按原解释扩回，是否恢复原位串/数值。

逻辑移位补零；算术右移复制补码符号位，常对应向负无穷取整的除 2^k ，不自动等于高级语言的有符号除法。左移低位结果与乘 2^k 同余，但是否溢出仍取决于目标解释和被丢高位。

不变量

每次扩展、截断或移位都问：要保留的是位模式、无符号值、补码值，还是仅保留模结果？若答案不明确，后续标志判断没有语义基础。

4 乘法：完整乘积、部分积与目标宽度

两个 n 位无符号数的完整乘积最多需要 $2n$ 位；两个 n 位补码数的完整乘积也用 $2n$ 位承载。若只保留低 n 位，unsigned 与补码乘法的低位位串相同，但高位和溢出解释不同。

窄结果判据：unsigned 要求完整乘积高 n 位全零；补码要求完整乘积高 n 位等于低半部符号位的扩展。更一般地，先得到足宽精确积，再检查是否能无损压回目标格式。

4.1 移位加法与原码一位乘

无符号乘法把乘数每一位 q_i 选择的 $M \cdot 2^i$ 累加。序贯乘法器用部分积、乘数和被乘数移位，以时间换加法器/寄存器资源；并行阵列同时生成和压缩部分积，以面积与关键路径换吞吐。

原码一位乘先取符号异或，再对绝对值做无符号部分积；它不是补码 Booth 算法的同义词。题目给出联合寄存器时，必须先写每段宽度、移位方向和每拍不变量，不能跨教材套门数或节拍数。

4.2 Booth 的生成逻辑

Radix-2 Booth 用相邻位 (Q_0, Q_{-1}) 压缩连续的 1: 01 加 M , 10 减 M , 00/11 不变，然后对联合寄存器算术右移。它利用

$$00111100_2 = 01000000_2 - 00000100_2$$

把一串加法变为边界处加/减。若题目使用更高基 Booth、不同位对次序或不同寄存器布局，先声明版本；“Booth 对所有输入都更快”不是稳定结论。

5 除法：商余关系是最终不变量

除法的稳定正确性条件是

$$X = YQ + R, \quad |R| < |Y|,$$

余数符号和商的取整方向服从题设/ISA。恢复余数法在试减为负时恢复；不恢复余数法让当前余数符号决定下一拍加还是减，省去立即恢复，但结束时可能需要校正。

算法题先固定 partial remainder、quotient/dividend、divisor 和附加位布局。每拍写判断位、加减对象、移位方向和上商位；最终用商余关系回代。原码与补码算法、左移余数与右移除数版本不能混用同一口诀。

除数为零可由零检测逻辑发现，但架构结果可能是同步异常、规定值或其他语义。补码最小负数除以 -1 会产生超出同宽 signed 范围的商；“同位宽整数除法不溢出”是错误命题。

6 浮点：在非均匀格点上表示实数近似

普通二进制规格化数为

$$x = (-1)^s (1.f)_2 2^{E-Bias}.$$

符号选方向，阶码选尺度，尾数选该尺度上的格点。阶码全 0 或全 1 必须分支处理 zero、subnormal、infinity 和 NaN，不能套普通隐藏位公式。

subnormal 使用 $0.f$ 和最小正常指数，提供渐进下溢；infinity 与 NaN 的传播、比较和异常细节以标准/题设为准。把整个 IEEE 754 编码称作“原码”会掩盖偏置阶码、隐藏位和特殊编码。

相邻可表示数的绝对间距随指数增长，因此“大数精度更高”与“有效位数固定”不是一回事：有效相对精度近似稳定，但绝对间距变大。多数十进制小数在二进制中无限展开，输入编码时已经发生第一次舍入。

7 浮点运算链与舍入证据

加减法的生成链是

Classify → Align → Significand Operation → Normalize → Round → Encode/Signal.

对阶右移较小尾数，sticky 位应记录所有被移出低位的 OR；guard、round、sticky 与保留末位共同决定舍入。round-to-nearest, ties-to-even 下，恰好中点时让保留结果最低位为偶数；其他舍入模式必须另算。

乘除先处理符号和阶码，再算有效数，规格化、舍入并检查 exponent range。overflow 与 underflow 是目标格式边界；inexact 表示精确结果不在目标格点上。只看阶差达到尾数位数就断言小数完全无影响，可能漏掉 GRS 对最终舍入的作用。

三个浮点反例

$0.1 + 0.2$ 涉及输入编码、运算和结果编码的多次有限映射；小量对阶后主体尾数为零，sticky 仍可能改变舍入；浮点加法不满足结合律，括号会改变中间舍入点。

8 五列概念边界

概念 A	≠ 概念 B	真正区别与题面信号	混淆后果
bit pattern	≠ value	存储对象 vs 解释后的数学对象	同位串误判
carry	≠ overflow	unsigned 出位 vs signed 越界	标志互推
sign extension	≠ zero extension	保补码值 vs 保 unsigned 值	扩宽变值
full product	≠ narrow result	足宽精确积 vs 目标寄存器保留部分	漏乘法溢出
divide-by-zero detect	≠ ISA exception	电路条件 vs 软件可见契约	从门电路推出 OS 行为
rounding	≠ truncation	映射到最近/方向格点 vs 直接丢低位	浮点结果错一 ULP

9 做题控制协议

- 1. 写位宽、解释、运算和目标格式；
- 2. 先写可表示集合与精确数学结果；
- 3. 用足够中间位执行扩展、加减、部分积或商余更新；
- 4. 明确第一次丢信息的位置与舍入/截断规则；

- 5. 分开判断 carry、overflow、zero、sign、inexact 等状态；
- 6. 乘除用积回算或 $X = YQ + R$ ；浮点用分类、数量级和反向解码校验；
- 7. 最后才套 ISA 对标志、异常、饱和或返回值的契约。

压缩成第一动作

看到表示或运算题，先问：这串位按什么格式解释？精确结果在哪一步被压回多少位，丢掉了什么证据？

10 全科坐标与跨册交接

Map Coordinate: ISA State / Semantics

CO-01 负责把无语义的位串解释成给定位宽下的值、结果和信息损失证据。它向 CO-02/03 输出可消费的 typed value、宽度、符号扩展/舍入结果与标志候选；异常是否成为软件可见事件仍由 ISA 契约决定。

本册的局部成本包括运算位宽、加法器/乘除迭代次数、关键路径、舍入逻辑和精度损失。它不直接给出程序总时间，性能题要把这些参数交给 ‘IC x CPI x clock period’ 的全局成本口径。

11 考纲映射与来源处理

考点	本册机制	接口
整数表示与运算	位权、补码、扩展、标志与目标位宽	CO-02/03
乘法器与 Booth	部分积、联合寄存器、相邻位重编码	CO-03 Use
除法与溢出	商余不变量、恢复/不恢复、边界检测	ISA exception
IEEE 754	分类、对阶、规格化、GRS 舍入、特殊值	ISA rounding/status

一句话

有限硬件的核心不是“把数学算错”，而是按明确格式把无限或足宽结果投影回有限集合；正确答案必须同时给出保留结果、丢失信息和其软件可见解释。

本册吸收旧《定点小数》《IEEE》《浮点数》及十二张定点/浮点/乘除卡片；并行阵列只保留空间换时间的 Extension，具体 Booth/不恢复余数变体服从题设，空白《运算电路》不产生知识更新。